

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON

CLASE 06 • NIVEL 1 - PRINCIPIANTE

Clase 06: Diccionarios y Conjuntos (Sets)

«Diccionarios como un Casillero con Llaves Únicas»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 06** del **Curso 1: Fundamentos Básicos de Python**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la estructura clave-valor, tablas hash, acceso $O(1)$ y unicidad en sets.

Competencia Práctica: Estructurar y consultar información compleja mediante diccionarios y métodos `.get()` y `.items()`.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 06: Diccionarios y Conjuntos (Sets)

Los diccionarios son colecciones asociativas basadas en pares clave-valor que permiten accesos ultra rápidos.

☀ Metáfora Central: Diccionarios como un Casillero con Llaves Únicas

Un diccionario es como un casillero: con tu llave (clave) abres instantáneamente el compartimento (valor).

Principios Teóricos y Modelo Mental

Las claves deben ser objetos inmutables y hashables (strings, números, tuplas).

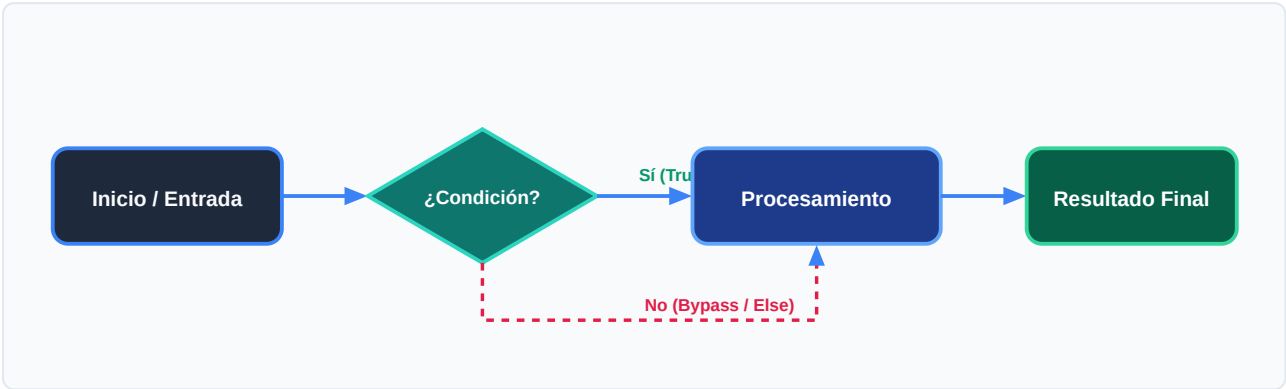
Los conjuntos (sets) son colecciones no ordenadas de elementos únicos.

⚡ Regla de Oro en Python

Usa siempre `diccionario.get('clave', default)` para evitar excepciones `KeyError`.

Arquitectura y Movimiento de Datos en Memoria

Hashing de claves, mapeo en tabla interna y operaciones de conjuntos.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Cálculo del hash mediante hash(key).	Hash entero generado.
2. Evaluación	Indexación en la tabla hash interna.	Ubicación del bucket.
3. Transformación	Recuperación del puntero al valor.	Acceso O(1).
4. Retorno / Salida	Iteración sobre items() o keys().	Vista dinámica generada.

Visualización Mental

Imagina los sets como un filtro que rechaza automáticamente cualquier duplicado.

Código de Demostración en Python 3.10+

Modelado de usuario y consulta segura:

main.py (Python 3.10+)

PEP 8 Compliant

```
usuario = {
    "id": 101,
    "nombre": "Carlos Ruiz",
    "roles": {"admin", "editor"},
    "activo": True
}

email = usuario.get("email", "sin_correo@empresa.com")
print(f"Usuario: {usuario['nombre']} | Email: {email}")
```

Análisis de Ingeniería del Código

Uso de .get() seguro con valor por defecto y set para roles sin duplicados.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Acceder directamente con corchetes a una clave inexistente rompe la ejecución.

⚠ Gotcha Frecuente (Trampa de Principiante)

Hacer `data['no_existe']` lanza `KeyError` en lugar de devolver `None`.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
data = {'a': 1}
val = data['b'] # ❌ KeyError
```

✅ Patrón Correcto

```
data = {'a': 1}
val = data.get('b', 0) # ✅ Seguro
```

🛡 Consejo de Resiliencia en Producción

Utiliza `collections.defaultdict` para inicializar contadores automáticos.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 06: Diccionarios y Conjuntos (Sets)**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Diccionarios como un Casillero con Llaves Únicas
2. Regla de Oro	Usa siempre <code>diccionario.get('clave', default)</code> para evitar excepciones <code>KeyError</code> .
3. Gotcha Crítico	Hacer <code>data['no_existe']</code> lanza <code>KeyError</code> en lugar de devolver <code>None</code> .
4. Buenas Prácticas	Utiliza <code>collections.defaultdict</code> para inicializar contadores automáticos.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea una función que reciba un texto y cuente la frecuencia de cada palabra con un diccionario.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.